50

PRACTICAL INTERVIEW GUIDE

LLM Interview Questions

Master Core Architectures & Production Trade-offs

- 🧠 Architecture & Attn
- ⚡ KV Cache & Speed
- 🔍 RAG & Context
- 🔧 PEFT, LoRA & Tuning
- ⚖️ RLHF, DPO & Alignment
- 📊 Evals, Cost & Serving

RESNUR AI

Short, precise answers for quick revision →

What is a Language Model? How does it work?

ANSWER

A Language Model (LM) predicts the next most likely token based on previous tokens.

How it works:

- Trained on massive text using self-supervised learning.
- Learns grammar, facts, reasoning, and language patterns by predicting missing or next tokens.

During inference:

→ Tokenize input

→ Run through transformer layers

→ Generate logits

→ Sample next token

→ Repeat until completion

LANGUAGE MODEL SELF-SUPERVISED

Base Model vs Instruction-Tuned Model

ANSWER

Base Model

- Predicts the next token.
- Continues text instead of following instructions.
- Best for domain-specific fine-tuning.

Instruction-Tuned Model

- Fine-tuned on instruction-response datasets.
- Further aligned using RLHF or Constitutional AI.
- Follows user instructions conversationally.

Most production applications use instruction-tuned models.

BASE MODEL INSTRUCTION-TUNED

REVISE & PREPARE FOR LLM ROLES

Encoder-only vs Decoder-only vs Encoder-Decoder

ANSWER

Architecture	Best For
Encoder-only (BERT)	Classification, NER, semantic search, extractive QA
Decoder-only (GPT)	Chat, text generation, code completion
Encoder-Decoder (T5/BART)	Translation, summarization, abstractive QA

Today, most frontier LLMs (GPT, Claude, Gemini) use decoder-only architectures.

ENCODER-ONLY DECODER-ONLY ENCODER-DECODER

REVISE & PREPARE FOR LLM ROLES

How does Autoregressive Generation work?

ANSWER

The model generates one token at a time, with each token conditioned on all previous ones.

Steps:

→ Tokenize input

→ Generate embeddings

→ Run through transformer layers

→ Compute logits

→ Sample next token

→ Append and repeat

Why it's slow: Each new token requires a new forward pass.

Optimization: KV Cache stores previous computations to speed up generation.

AUTOREGRESSIVE INFERENCE LATENCY

What is Perplexity?

ANSWER

Perplexity measures how well an LLM predicts text.

- Lower perplexity = Better predictions
- Based on the model's average probability assigned to the correct next token.

Why it's useful:

→ Fast and inexpensive evaluation.

→ Correlates with language modeling quality.

Limitation: Doesn't measure reasoning, factual accuracy, or helpfulness.

PERPLEXITY EVALUATION

REVISE & PREPARE FOR LLM ROLES

Pretraining vs Fine-Tuning

ANSWER

Pretraining

- Trains on massive unlabeled datasets.
- Learns general language understanding.
- Expensive and compute-intensive.

Fine-Tuning

- Continues training on task-specific data.
- Faster and much cheaper.
- Adapts an existing model to a specific use case.

Most teams fine-tune existing models using LoRA or QLoRA instead of pretraining from scratch.

PRETRAINING FINE-TUNING

What is Tokenization?

ANSWER

Tokenization converts text into tokens that an LLM can process.

Common methods:

- Word Tokenization – Split by words.
- Character Tokenization – Split into characters.
- Subword Tokenization – Most common approach today.

Popular algorithms:

→ BPE – Used by GPT.

→ WordPiece – Used by BERT.

→ SentencePiece – Used by LLaMA and T5.

Different models tokenize text differently, affecting context length, cost, and performance.

TOKENIZATION BPE SUBWORD

REVISE & PREPARE FOR LLM ROLES

What is the Context Window?

ANSWER

The context window is the maximum number of tokens an LLM can process at once.

If exceeded:

→ Older models truncate input.

→ Modern systems use chunking, sliding windows, or summarization.

Important: Larger context doesn't always improve performance due to the Lost in the Middle problem.

For RAG: Retrieved documents, prompts, and chat history must all fit within the context window.

CONTEXT WINDOW LOST IN THE MIDDLE

Beam Search vs Greedy Decoding

ANSWER

Greedy Decoding

- Picks the highest-probability token each step.
- Fast but may miss better overall sequences.

Beam Search

- Keeps multiple candidate sequences (beams).
- Produces more coherent results.
- More computationally expensive.

Modern chat applications usually prefer temperature + Top-P sampling over beam search.

GREEDY DECODING BEAM SEARCH

REVISE & PREPARE FOR LLM ROLES

Top-K vs Top-P vs Temperature

ANSWER

These techniques control how the next token is selected.

Temperature

- Controls randomness.
- Low → Deterministic
- High → More creative

Top-K

- Samples from the top K tokens.
- Fixed-size candidate pool.

Top-P (Nucleus Sampling)

- Samples from the smallest set of tokens whose cumulative probability reaches P.
- Adaptive and commonly preferred.

Typical settings: Temperature 0 for factual tasks, 0.7 + Top-P 0.9 for chat, and 1.0 + Top-P 0.95 for creative writing.

TOP-K TOP-P TEMPERATURE

What is Few-Shot Prompting? Why does it work?

ANSWER

Few-shot prompting provides 2–10 examples in the prompt before the actual query—without changing model weights.

Why it works:

- LLMs recognize patterns from examples (in-context learning).
- It guides the model on what to do and how to format the output.

Best practices:

→ Use high-quality, diverse examples.

→ Keep formatting consistent.

→ Recent examples have stronger influence.

Few-shot usually outperforms zero-shot for structured tasks like extraction, classification, and code generation.

FEW-SHOT IN-CONTEXT LEARNING

What is In-Context Learning (ICL)?

ANSWER

In-Context Learning (ICL) is the ability of an LLM to learn a task from examples within the prompt, without updating its weights.

How it works:

- Reads examples.
- Infers the pattern.
- Applies it to the new input.

Key points:

→ No training or gradient updates.

→ Emerges in large-scale models.

→ Limited by context size and example quality.

IN-CONTEXT LEARNING EMERGENCE

REVISE & PREPARE FOR LLM ROLES

What is Catastrophic Forgetting?

ANSWER

Catastrophic forgetting occurs when fine-tuning causes a model to lose previously learned knowledge.

Example: Fine-tuning for medical QA reduces coding performance.

Prevention techniques:

- LoRA / PEFT (freeze base model)
- Elastic Weight Consolidation (EWC)
- Replay methods
- Lower learning rates

LoRA is the most common production solution because it preserves the base model's knowledge.

CATASTROPHIC FORGETTING LORA

Zero-Shot vs Few-Shot vs Fine-Tuning

ANSWER

Method	Best Use Case
Zero-shot	General tasks without examples
Few-shot	Structured tasks needing examples or formatting
Fine-tuning	Domain-specific behavior and consistent performance

Trade-off:

→ Zero-shot → Lowest cost

→ Few-shot → Better accuracy

→ Fine-tuning → Best performance but requires data and compute

ZERO-SHOT FEW-SHOT FINE-TUNING

What is Emergent Behavior in LLMs?

ANSWER

Emergent behaviors are capabilities that appear only when models reach a certain scale.

Examples:

- Chain-of-Thought reasoning
- In-Context Learning
- Code generation
- Arithmetic
- Analogical reasoning

Why it matters:

→ These abilities aren't explicitly trained.

→ They're difficult to predict from smaller models.

→ Some emergent behaviors have important AI safety implications.

EMERGENT CAPABILITY MODEL SCALE

Explain Self-Attention Mathematically

ANSWER

Self-attention lets each token focus on the most relevant tokens in the sequence.

Steps:

→ Compute Query (Q), Key (K), and Value (V) vectors.

→ Calculate attention scores:

$$\text{Attention} = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

- Multi-head attention repeats this process across multiple heads to learn different relationships.
- The $\sqrt{d_k}$ scaling keeps attention scores stable during training.

SELF-ATTENTION MATH MULTI-HEAD

REVISE & PREPARE FOR LLM ROLES

What is Positional Encoding?

ANSWER

Transformers don't understand word order by default, so positional encoding adds position information to tokens.

Common methods:

- Sinusoidal Encoding – Fixed mathematical encoding.
- Learned Embeddings – Trainable position vectors.
- RoPE (Rotary Position Embeddings) – Standard for modern LLMs.
- ALiBi – Position-based attention bias.

RoPE is widely used because it generalizes well to longer contexts.

POSITIONAL ENCODING ROPE ALIBI

What is KV Cache?

ANSWER

KV Cache stores previously computed Key and Value vectors during text generation.

Benefits:

→ Avoids recomputing previous tokens.

→ Reduces inference complexity from repeated computation.

→ Significantly speeds up autoregressive generation.

Trade-off:

- Uses additional GPU memory.
- Optimizations like Paged Attention (vLLM) improve efficiency.

KV CACHE VLLM PAGED ATTENTION

REVISE & PREPARE FOR LLM ROLES

What is Flash Attention?

ANSWER

Flash Attention is an optimized attention algorithm that reduces memory usage and speeds up inference.

Benefits:

- Uses O(N) memory instead of $O(N^2)$.
- 2–4× faster than standard attention.
- Enables efficient long-context training and inference.

Flash Attention is now the standard implementation for modern LLMs.

FLASH ATTENTION | SRAM / HBM | SPEED

What is Grouped Query Attention (GQA)?

ANSWER

Grouped Query Attention (GQA) reduces KV-cache size by allowing multiple attention heads to share Keys and Values.

Comparison:

- Multi-Head Attention: Separate K & V for every head.
- MQA (Multi-Query): All heads share one K & V.
- GQA (Grouped-Query): Groups of heads share K & V.

Why it matters:

→ Lower memory usage.

→ Higher inference throughput.

→ Used in LLaMA 2/3, Mistral, and Gemma.

GQA | MQA | MHA

What is Speculative Decoding?

ANSWER

Speculative decoding speeds up inference using a small draft model and a large target model.

How it works:

→ Small model generates candidate tokens.

→ Large model verifies them in parallel.

→ Correct tokens are accepted; incorrect ones are regenerated.

Benefits:

- 2–3× faster inference.
- Same output quality.
- Used in production by Gemini and Claude.

SPECULATIVE DECODING DRAFT MODEL

What is RLHF? Explain the process.

ANSWER

RLHF (Reinforcement Learning from Human Feedback) aligns LLMs to be more helpful, harmless, and honest.

Stages:

1. Supervised Fine-Tuning (SFT)

→ Train the base model on human-written instruction-response pairs.

2. Reward Model Training

→ Human raters rank multiple responses.

→ Train a reward model to predict human preferences.

3. PPO Optimization

→ Use the reward model to optimize the LLM with Proximal Policy Optimization (PPO).

→ A KL penalty prevents the model from drifting too far from the SFT model.

Result: An aligned assistant like ChatGPT that better follows user intent.

RLHF SFT PPO REWARD MODEL

What is DPO? How is it different from RLHF?

ANSWER

DPO (Direct Preference Optimization) aligns LLMs directly from preference data—without a reward model or reinforcement learning.

How it works:

- Uses preferred vs rejected response pairs.
- Increases the probability of preferred responses through a single loss function.

DPO	RLHF
No reward model	Requires reward model
No PPO	Uses PPO
Simpler & more stable	More complex pipeline
Faster training	Better for some complex reasoning tasks

DPO is becoming a popular alternative for aligning open-source LLMs.

DPO RLHF PREFERENCE OPTIMIZATION

What is Instruction Tuning?

ANSWER

Instruction tuning fine-tunes a pretrained model using instruction-response examples.

Before: Model completes text. Doesn't reliably follow instructions.

After: Understands and follows natural language instructions across many tasks.

Popular datasets:

- FLAN • Alpaca
- OpenHermes • UltraChat

Instruction tuning significantly improves zero-shot performance on unseen tasks.

INSTRUCTION TUNING ZERO-SHOT

What are Scaling Laws?

ANSWER

Scaling laws describe how LLM performance improves as model size, training data, and compute increase.

Key findings:

- Larger models generally perform better.
- Chinchilla Scaling showed that balancing model size and training data is more effective than simply increasing parameters.

Impact:

→ Modern LLMs use better model-to-data ratios.

→ Smaller, well-trained models can outperform much larger undertrained ones.

SCALING LAWS CHINCHILLA SCALING

What is Mixture of Experts (MoE)?

ANSWER

Mixture of Experts (MoE) activates only a small subset of expert networks for each input token.

How it works:

→ A router selects the Top-K experts (usually 2).

→ Only those experts process the token.

Benefits:

- Higher model capacity • Lower computation per token
- Better efficiency at scale

Challenges: Load balancing across experts, more complex distributed training/serving.

MoE enables larger models without proportional increases in inference cost.

MOE ROUTER NETWORK SPARSE MODEL

What is Context Window Extension?

ANSWER

Context window extension increases the amount of text an LLM can process beyond its original limit.

Common techniques:

- RoPE Scaling – Extends rotary positional embeddings.
- YaRN – Fine-tunes models for longer contexts.
- LongLoRA – Efficient long-context fine-tuning.
- ALiBi – Uses attention bias for better length generalization.

Best practice: Use models that already support long contexts (e.g., Llama 3 or Mistral) instead of extending them yourself unless necessary.

CONTEXT EXTENSION ROPE SCALING YARN

REVISE & PREPARE FOR LLM ROLES

What is Retrieval-Augmented Generation (RAG)? When does it fail?

ANSWER

RAG improves LLM responses by retrieving relevant external documents, reducing hallucinations and outdated knowledge.

Common failure modes:

- Retrieval failure: Wrong chunks retrieved due to poor chunking, weak embeddings, or vocabulary mismatch. Fix: Hybrid search, better chunking, reranking.
- Context conflict: Retrieved documents contradict each other. Fix: Source filtering, date-based ranking, conflict-aware prompts.
- Lost in the middle: Relevant information is buried in long context. Fix: Rerank and place important chunks strategically.
- Faithfulness failure: Model ignores retrieved context and answers from memory. Fix: Strong grounding prompts, lower temperature.
- Query mismatch: Complex questions retrieve incomplete evidence. Fix: Query decomposition, multi-hop retrieval, GraphRAG.

Interview Tip: Understanding RAG failures is often more important than just explaining RAG.

RAG FAILURES HYBRID SEARCH GROUNDING

What is Hallucination? What are the different types?

ANSWER

A hallucination is when an LLM generates fluent but incorrect or unsupported information.

Types:

- Intrinsic: Contradicts the provided context (most critical in RAG).
- Extrinsic: Adds unsupported information beyond the source.
- Factual: States incorrect real-world facts.
- Temporal: Uses outdated information as current.
- Faithfulness: Adds details not present in the original document (common in summarization).

Why it happens: Models optimize for plausibility, not truth. Training data contains errors and outdated facts. No built-in fact-checking mechanism.

Mitigation: RAG, lower temperature, uncertainty prompts, output validation, and LLM-as-Judge.

HALLUCINATIONS NLI GROUNDING LLM-AS-JUDGE

What is Prompt Injection? Types & Defenses

ANSWER

Prompt injection is an attack where malicious input manipulates an LLM into ignoring its intended instructions.

Types:

- Direct: User explicitly overrides the system prompt.
- Indirect: Malicious instructions hidden in external content (e.g., web pages, documents).
- Multi-turn: Gradually changes model behavior across multiple conversations.

Defenses:

→ Input validation → Prompt hardening

→ Privilege separation → Least-privilege tool access

→ Output monitoring → Sandboxing agent actions

PROMPT INJECTION THREAT DEFENSE SANDBOXING

REVISE & PREPARE FOR LLM ROLES

Parametric vs Non-Parametric Knowledge

ANSWER

Parametric Knowledge

- Stored in model weights during training.
- Always available but static.
- Can be outdated and requires retraining to update.

Non-Parametric Knowledge

- Provided at inference via RAG, documents, or conversation history.
- Dynamic, verifiable, and easily updated.

Key takeaway: Modern LLM apps should treat the model as a reasoning engine, while using RAG as the knowledge source.

PARAMETRIC NON-PARAMETRIC

What is Chain-of-Thought (CoT) Prompting?

ANSWER

Chain-of-Thought (CoT) encourages the model to reason step by step, improving multi-step problem solving.

Variants:

- Zero-shot CoT: "Think step by step."
- Few-shot CoT: Include reasoning examples.
- Self-consistency: Generate multiple reasoning paths and vote.
- Tree of Thoughts (ToT): Explore and evaluate multiple reasoning branches.
- Least-to-most: Solve simple sub-problems before complex ones.

Best for: Math, logic, planning, and reasoning tasks.

Less useful for: Simple retrieval or classification.

CHAIN-OF-THOUGHT TREE OF THOUGHTS REASONING

What is Function Calling / Tool Use?

ANSWER

Function calling lets LLMs invoke external tools instead of relying only on generated text.

How it works:

→ Define tools with a JSON schema.

→ LLM selects the appropriate tool.

→ Application executes it.

→ Results are returned to the LLM.

→ LLM generates the final response.

Common use cases: Web search, database queries, API calls, calculators, code execution, file operations.

Function Calling vs Agents:

- Function Calling: Single call → result → response.
- Agents: Multiple tool calls in a reasoning loop until the goal is achieved.

FUNCTION CALLING TOOL USE AI AGENTS

What is Structured Output Generation?

ANSWER

Structured output ensures LLM responses follow formats like JSON, XML, or custom schemas for reliable parsing.

Challenges: Invalid JSON, missing fields, incorrect data types, formatting errors.

Ways to improve reliability (best → good):

1. Native structured output mode (most reliable constrained decoding).
2. Schema validation (e.g., Pydantic/Instructor library).
3. Few-shot formatting examples in the prompt.
4. Automatic retry after validation failure.
5. Constrained decoding frameworks (e.g., Outlines, Guidance).

Best Practice: Use native structured output with schema validation for production systems.

STRUCTURED OUTPUT JSON SCHEMA PYDANTIC

What is the Lost in the Middle Problem?

ANSWER

The Lost in the Middle problem occurs when LLMs perform better on information at the beginning and end of long contexts but often ignore important details in the middle.

Example: In RAG, the most relevant chunk may be retrieved correctly but overlooked if it's buried in the middle.

Why it happens:

- Recency bias: Focus on recent tokens.
- Primacy bias: More attention to the beginning.

Mitigation:

→ Place the most relevant chunks first (and second most relevant last).

→ Retrieve fewer, higher-quality chunks.

→ Rerank retrieved documents.

→ Use contextual compression or multi-step retrieval.

LOST IN THE MIDDLE RERANKING CONTEXT COMPRESSION

How would you reduce LLM inference costs in production?

ANSWER

Reduce costs by optimizing both the model and serving pipeline.

Key strategies:

1. Use the smallest model that meets quality requirements.
2. Apply INT8/INT4 quantization.
3. Use semantic caching for repeated queries.
4. Batch requests to improve GPU utilization.
5. Optimize prompts to reduce token usage.
6. Use speculative decoding for faster generation.
7. Route simple queries to smaller models (model cascading).
8. Optimize KV-cache with solutions like vLLM.

COST REDUCTION QUANTIZATION MODEL CASCADE

How do you evaluate LLM outputs at scale?

ANSWER

No single metric is enough—combine automated and human evaluation.

Common methods:

- Automated metrics: BLEU, ROUGE, BERTScore, Perplexity.
- LLM-as-Judge: Scores relevance, correctness, coherence, and safety.
- Reference-based: Exact Match, F1, semantic similarity.
- Preference evaluation: Compare two responses.
- Task-specific metrics: Unit tests (code), RAGAS (RAG).
- Production monitoring: Track user feedback, corrections, and abandonment.

Best practice: Combine automated metrics, LLM judges, and periodic human review.

LLM EVALUATION LLM-AS-JUDGE SCALE EVALS

REVISE & PREPARE FOR LLM ROLES

What is Constitutional AI? How does it work?

ANSWER

Constitutional AI aligns models using a predefined set of principles instead of relying only on human feedback.

Process:

→ Model generates a response.

→ It critiques the response using constitutional principles.

→ It revises the response.

→ AI-generated preferences are used for further alignment.

Advantages: Reduces dependence on human labeling, highly scalable and consistent, principles are transparent and easy to update.

CONSTITUTIONAL AI | RLAIF | AI FEEDBACK

RAG vs Fine-Tuning vs Long Context: When to use each?

ANSWER

RAG

- Dynamic knowledge, source citations, large document collections, frequently changing data.

Fine-Tuning

- Domain-specific behavior, consistent tone/style, specialized tasks, prompt token reduction.

Long Context

- Small knowledge base fits in context, holistic reasoning across documents, unpredictable retrieval needs.

In practice: Most production systems combine all three.

RAG VS FT | CONTEXT SELECTION | LLM STRATEGY

How would you build an LLM evaluation pipeline?

ANSWER

1. Define quality metrics – correctness, helpfulness, safety.
2. Build an evaluation dataset – include normal, edge, and adversarial cases.
3. Choose evaluation methods – Exact Match, semantic similarity, unit tests, LLM-as-Judge, human review.
4. Automate evaluation – run on every model or prompt change.
5. Set thresholds – detect regressions and trigger rollbacks.
6. Monitor production – track user feedback and real-world performance.
7. Human review – periodically audit outputs to calibrate automated metrics.

Key takeaway: Build the evaluation pipeline before deploying the LLM system.

EVAL PIPELINE GROUND TRUTH REGRESSION TESTS

What is Model Alignment? What are the key challenges?

ANSWER

Model Alignment ensures AI behaves according to human values and intent—not just literal instructions.

Key challenges:

- Reward hacking: Optimizes the reward instead of the actual goal.
- Specification gaming: Follows the letter, not the intent, of instructions.
- Goal misgeneralization: Learns shortcuts that fail in new scenarios.
- Scalable oversight: Humans can't reliably evaluate highly capable models.
- Deceptive alignment: Behaves well during training but differently in deployment.

Current approaches: RLHF, Constitutional AI, interpretability, and scalable oversight. Alignment remains an active research area.

MODEL ALIGNMENT REWARD HACKING AI SAFETY

REVISE & PREPARE FOR LLM ROLES

What is Interpretability in LLMs? Why does it matter?

ANSWER

Interpretability (Explainability) helps us understand how and why an LLM makes decisions.

Common techniques:

- Attention visualization – Shows attention patterns (not true explanations).
- Probing classifiers – Reveal information stored in hidden layers.
- Activation patching – Identifies components responsible for outputs.
- Sparse autoencoders – Discover interpretable internal features.

Why it matters: AI safety, model debugging, trust in high-stakes applications, and understanding model behavior.

INTERPRETABILITY SPARSE AUTOENCODERS MECHANISTIC

What is Continual Learning?

ANSWER

Continual learning updates an LLM with new knowledge without forgetting existing capabilities.

Challenges: Standard fine-tuning causes catastrophic forgetting.

Approaches:

- LoRA adapters (parameter-efficient updates)
- Experience replay
- Elastic Weight Consolidation (EWC)
- Retrieval-Augmented Learning (RAG) instead of weight updates

In practice: Most production systems use RAG for knowledge updates rather than retraining.

CONTINUAL LEARNING WEIGHT CONSOLIDATION EWC

REVISE & PREPARE FOR LLM ROLES

How do you handle Sensitive Data & PII in LLM applications?

ANSWER

Protect sensitive data using multiple security layers.

Best practices:

1. Minimize data shared with the LLM.
2. Detect and redact PII before inference.
3. Use self-hosted models for highly sensitive workloads.
4. Encrypt data in transit and at rest.
5. Scan outputs for sensitive information.
6. Maintain audit logs and retention policies.
7. Apply differential privacy during fine-tuning when needed.

PII REDACTION DATA SECURITY COMPLIANCE

What is Model Watermarking?

ANSWER

Model watermarking embeds detectable patterns into AI-generated text to verify its origin.

Types:

- Statistical watermarking: Biases token selection in a detectable way.
- Soft watermarking: Invisible patterns robust to minor edits.

Challenges: Can be removed through paraphrasing or translation. No watermark is completely attack-proof.

Why it matters: Helps with AI content provenance, regulation, and misuse detection.

MODEL WATERMARKING PROVENANCE

What is PEFT? Compare LoRA, Prefix Tuning & Prompt Tuning

ANSWER

PEFT (Parameter-Efficient Fine-Tuning) adapts LLMs by training only a small subset of parameters.

Method	Summary
LoRA	Trains low-rank adapters. Best balance of quality and efficiency.
Prefix Tuning	Learns virtual tokens at every transformer layer.
Prompt Tuning	Learns only input prompt embeddings. Works best for very large models.

Interview Tip: LoRA is the most widely used PEFT method in production.

PEFT LORA PREFIX TUNING PROMPT TUNING

Open Source vs Proprietary LLMs

ANSWER

Proprietary Models	Open Source Models
Highest capability • Managed infrastructure • Regular updates • Pay-per-token	Full data control • Fine-tunable • Lower cost at scale • No vendor lock-in
Limited customization • Privacy / compliance concerns	Requires infrastructure expertise

Rule of thumb: Prototype with proprietary models, scale with open-source where appropriate.

OPEN SOURCE PROPRIETARY INFRASTRUCTURE

REVISE & PREPARE FOR LLM ROLES

How do you prevent Harmful LLM Outputs?

ANSWER

Use defense-in-depth instead of relying on a single safeguard.

Layers:

- Input filtering • Strong system prompts
- RLHF/Constitutional AI aligned models • Output moderation
- Retrieval filtering (for RAG) • Human review
- Red teaming • Rate limiting and anomaly detection

SAFEGUARDS MODERATION API RED TEAMING

What is Test-Time Compute?

ANSWER

Test-Time Compute (TTC) spends extra computation during inference to improve reasoning quality.

Techniques:

- Self-consistency • Best-of-N sampling
- Chain-of-Thought with verification • Monte Carlo Tree Search (MCTS)
- Process Reward Models (PRMs)

Benefit: Better reasoning at the cost of higher latency and compute.

TEST-TIME COMPUTE MCTS PRM

REVISE & PREPARE FOR LLM ROLES

Design an LLM-powered Enterprise Knowledge Base

ANSWER

1. Ingestion: Parse docs → Extract metadata → Chunk → Embed → Store in Vector DB.

2. Query Pipeline: Classify query → Hybrid retrieve → Rerank → Access control → Context assembly → Gen with citations.

3. Evaluation: Offline benchmarks (RAGAS), LLM-as-Judge, production monitoring.

4. Infrastructure: FastAPI, Redis, Kubernetes, Vector DB (Qdrant), PostgreSQL, Object storage.

5. Guardrails: PII detection, output filtering, access control, audit logs, rate limit, human escalation.

Interview Tip: Focus on the architecture flow (Ingestion → Retrieval → Generation → Evaluation → Guardrails) rather than listing every technology.

SYSTEM DESIGN ENTERPRISE RAG INFRASTRUCTURE

REVISE & PREPARE FOR LLM ROLES



Ready to Ace Your LLM Interview?

These 50 questions cover the critical technical, system design, and alignment concepts expected of LLM and AI Engineers today. Master the details, focus on trade-offs (latency, cost, accuracy), and show structured engineering thinking in your answers.

Follow ResnurAI
🌱 One critical LLM topic explained simply. Every single day.

Save this Post
📌 Save this — review it immediately before your next loop.

Share & Repost
♻️ Repost to share these questions with your engineering network.